Evidence-Based Linux Game Tuning Case Study: Kingdom Come: Deliverance 1 under Proton

Using scheduler-aware eBPF profiling to evaluate a CPU-affinity tuning hypothesis

1. Executive Summary

This case study evaluates whether `stutter`, an experimental Linux game-performance profiling and tuning toolkit, can collect evidence from a real Proton/Wine game workload, generate a scoped tuning hypothesis, and test whether the change supports better frame pacing. The tested CPU-affinity profile was plausible but was **not validated**: `baseline-online` had a lower primary diagnostic score than the tuned profile in all five paired A/B iterations. The archived run was paired but not counterbalanced: `baseline-online` was measured before the tuned profile in every iteration, so the result may include an order effect. The result is still useful as conservative non-validation evidence because it shows the tool can avoid false-positive tuning recommendations in a noisy real-world workload.

- Real game workload: Kingdom Come: Deliverance 1 under GE-Proton10-34.
- Repeatable 180-second Rattay route from the same save.
- Five formal baselines and five A/B iterations per profile.
- Tested CPU-affinity split: game/Wine on `1-5,7-11`, Gamescope/runtime on `0,6`.
- Result: not recommended on current evidence; the workflow worked.

The key answer is yes, within the limits of a preliminary case study: `stutter` demonstrated feasibility for an evidence-based workflow that can test a real Linux/Proton game tuning hypothesis. It collected evidence, generated a plausible CPU-affinity hypothesis, tested it, did not validate it, explained why that matters, and added explainability/follow-up analysis.

Primary evidence: `reports/kcd1-case-study/CASE_STUDY_SUMMARY.md`, curated copies in `evidence-bundle/kcd1/tuning_summary.json`, `evidence-bundle/kcd1/tuning_recommendation.json`, and `evidence-bundle/kcd1/profile-plan-summary.json`, with original raw paths preserved in the raw KCD1 release archive.

> **Artifact location note:** raw KCD1 run directories, tuning directories, MangoHud CSVs, setup logs, profile-plan artifacts, and exploratory stack comparison artifacts are no longer tracked in Git. They are preserved in the `stutter-kcd1-case-study-raw.tar.zst` asset attached to the `case-study-kcd1-2026` GitHub release. In command examples, `${KCD1_RAW}` denotes the root of the extracted raw archive's `reports/kcd1-case-study/`

directory. The normal repository checkout keeps the polished report, summary files, and curated `evidence-bundle/`.

2. Motivation

Linux gaming tuning advice is often anecdotal. Players change schedulers, launch flags, CPU affinity, compositor settings, Wine/Proton options, and graphics-driver flags, then judge the result from short play sessions or average FPS. That is not enough for a noisy open-world game such as Kingdom Come: Deliverance 1. Average FPS can look acceptable while frame pacing, 1% lows, p99 frame time, or scheduler delay still make the game feel uneven.

`stutter` is about evidence-based validation rather than magic automatic tuning. The aim is to determine whether a proposed tuning change is supported by repeated evidence under a controlled workload. A negative result is therefore useful. If a plausible tweak does not hold up across repeated A/B measurements, the tool should say so instead of turning a hypothesis into advice.

This case study exercises the full loop on a real game rather than on a synthetic benchmark: observation, diagnosis, scoped hypothesis, reversible tuning, repeated validation, uncertainty handling, and honest non-recommendation when the evidence is not strong enough.

3. Tool Overview

Only the parts of `stutter` that matter for this experiment are summarised here:

- eBPF runnable-latency profiling for scheduler-visible delay.
- Process-tree tracking for Proton, Wine, Gamescope, and related runtime tasks.
- MangoHud frame timing ingestion.
- GPU sampling.
- Data-quality checks.
- Advisor/fix-plan workflow.
- A/B tune/recommend pipeline.
- Profile explainability through `profile-plan` and `apply-profile --dry-run --explain`.

Experiment pipeline:

```
Record baseline -> Diagnose -> Generate hypothesis -> A/B tune -> Recommend /
needs retest / decline -> Interpret
                                        |
                                        v
                          Profile explainability follow-up
```

In this case study, the profile-plan explainability pass was added as a follow-up after the A/B run exposed the need to audit which rules matched which KCD1 threads. Future studies should run it before tuning.

The important property is that the tool does not stop at producing a plausible profile. It measures the profile against an online-baseline profile and can decline to recommend the tuning change.

4. Experimental Setup

Item	Value
Game	Kingdom Come: Deliverance 1
Platform	Steam + GE-Proton10-34
OS/session	Gentoo Linux, Sway/Wayland, Gamescope
CPU	Intel i5-10600K, 6 cores / 12 threads
GPU	AMD Radeon RX 9070 XT
Route	Repeatable Rattay route from the same save
Duration	180 seconds per recorded run
Baselines	5 formal baseline runs
A/B test	5 `baseline-online` + 5 tuned profile runs; paired but not counterbalanced
Frame capture	MangoHud CSV ingested by `stutter`
Main variable	CPU-affinity profile only

The shader cache was pre-warmed. The main experiment intentionally excluded the author's larger personal optimized launch configuration: no RADV experimental flags, no FSR/FSR4, no gamemode, no mimalloc, and no forced Wine CPU topology. Gamescope and MangoHud logging remained part of the measurement setup. The Steam launch used `+exec user.cfg`; the archived `user.cfg` is treated as part of the workload configuration rather than as the tuning variable.

The formal recordings used explicit `--tree-pid` targeting for the KCD1/Gamescope process tree. This should be presented as explicit process-tree targeting, not as a foreground-window auto-detection demonstration.

Setup evidence: setup notes, system information, and KCD1 config snapshots preserved under `${KCD1_RAW}/setup/` in the raw release archive.

> Some archived metadata records include kernel build strings containing `root@localhost`. This is the kernel image build identity embedded in `uname`/kernel-version metadata, not evidence that `stutter` itself was run as the root user for the benchmark route.

5. Baseline Findings

The five formal baseline runs passed the basic validity checks: each ran for about 180 seconds, stopped because `max_duration_reached`, ingested MangoHud frame data, used `monotonic_observed` frame timestamp alignment, and reported `Medium` data quality.

Run	Frames	Median frametime	P99	Max	Frame-pacing outliers (~33ms / 2x median)	Data quality
baseline-01	8,833	17.725ms	51.008ms	562.266ms	1,347	Medium
baseline-02	7,744	21.276ms	49.712ms	272.166ms	1,354	Medium
baseline-03	9,261	16.309ms	49.085ms	563.179ms	1,303	Medium
baseline-04	7,421	22.811ms	46.778ms	84.384ms	1,229	Medium
baseline-05	7,607	22.884ms	44.788ms	87.387ms	1,098	Medium

Baseline runs were valid but noisy. Median frametime varied noticeably across runs, while p99/tail behavior remained consistently problematic. This supports the case-study motivation: real game workloads can be too noisy for one-off tuning claims.

The evidence showed scheduler-visible tail latency and frame-pacing outliers. It does not show that scheduler delay was the only source of KCD1 stutter. The appropriate claim is narrower: the baseline data contained enough scheduler and frame evidence to justify a scoped tuning hypothesis and repeated A/B testing.

Baseline evidence: formal baseline analyses, postcheck files, and MangoHud CSVs preserved under `${KCD1_RAW}/runs/` and `${KCD1_RAW}/mangohud/` in the raw release archive.

6. Diagnostic Score Explanation

`diagnostic_raw_score_total` is an internal weighted penalty score used by `stutter` to compare runs under the same scenario and measurement settings. Lower is better. It is not FPS and it is not a general performance unit. In the frame-aware comparison path used here, it combines scheduler-latency threshold counts for relevant game/runtime classes with frame-time tail counts. Larger values mean more or worse scheduler/frame-pacing outliers during the measured window.

Simplified weighting:

```
scheduler component:
  over_5ms * 100
+ over_2ms * 20
+ over_1ms
```

```
frame component:
  frame_over_50ms * 100
+ frame_over_33ms * 20
+ frame_over_16ms
```

This score is useful inside this controlled case study because the same game, route, measurement window, and analysis path are used for the compared profiles. It should not be treated as a standalone unit that can be compared across unrelated systems or games.

Score evidence: `reports/kcd1-case-study/CASE_STUDY_SUMMARY.md` and the curated copy in `evidence-bundle/kcd1/tuning_summary.json`; the original raw file is preserved under `${KCD1_RAW}/tune/`.

7. Hypothesis and Profile

The advisor generated a plausible CPU-placement hypothesis: reserve the core-0 SMT pair for Gamescope/runtime work and place KCD1/Wine/game threads on the remaining CPUs.

The tested profile was:

Rule	Match	Affinity	Intended effect
0	`match_comm = ["Main"]`	`1-5,7-11`	Move KCD process threads
1	`Game`, `GameHelper`, `WineServer`	`1-5,7-11`	Move Wine/game helpers
2	`GameScope`, `Compositor`, `Launcher`, `SteamRuntime`	`0,6`	Reserve core-0 SMT pair for presentation/ runtime

Profile rules are first-match-wins. `match_comm` checks both `task.comm` and `process_comm`, so `match_comm = ["Main"]` can match worker threads whose own names are `RenderThread`, `ClothingRaycast`, `dxvk-submit`, and similar because they belong to a process whose `process_comm` is `Main`.

That matching behavior is important. It means the broad rule was capable of moving the major KCD worker threads, but it also means the profile compressed the game side onto 10 logical CPUs rather than keeping access to all 12 online CPUs.

Profile evidence: profile, advisor, and fix-plan artifacts preserved under `${KCD1_RAW}/profiles/` and the raw archive root; the compact profile-plan summary is curated in `evidence-bundle/kcd1/profile-plan-summary.json`.

8. Profile Explainability

The profile-plan artifact is one of the strongest additions to the case study. It shows what the tuned profile would do before applying it:

Item	Count
Snapshot tasks	181
Matched tasks	114
Pending affinity changes	114
Rule 0 matched tasks	88
Rule 1 matched tasks	25
Rule 2 matched tasks	1

Rule 0 caught many important KCD/DXVK/Wine-side worker threads through the broad `process_comm = "Main"` match, including `RenderThread`, `ClothingRaycast`, `Streaming Async`, `dxvk-submit`, `dxvk-cs`, audio, streaming, shader, physics, and job-system threads. Rule 1 matched remaining Wine/game helpers, including `winedevice.exe`, `steam.exe`, and `wineserver`. Rule 2 matched the Gamescope-side task.

The tuned profile did not fail simply because it missed the important KCD worker threads. The explainability artifact shows that those threads were matched and would have been moved. That makes the result more interesting: a plausible profile that did catch relevant tasks still did not win under repeated A/B measurement.

Explainability evidence: `${KCD1_RAW}/profiles/kcd1-affinity-profile-plan.txt`, `${KCD1_RAW}/profiles/kcd1-affinity-profile-plan.json`, and `${KCD1_RAW}/profiles/kcd1-affinity-profile-plan-summary.json`.

9. A/B Tuning Result

The paired A/B tuning run, `kcd1-affinity-02`, tested both profiles with five valid measured iterations each. Lower diagnostic score is better. This run was paired but not counterbalanced: `baseline-online` was measured before the tuned profile in every iteration, so profile effects may be confounded with measurement order.

Iteration	Baseline-online score	Tuned profile score	Delta	Lower score
1	19,579	32,052	+63.7%	baseline
2	21,533	34,566	+60.5%	baseline
3	16,643	44,845	+169.5%	baseline
4	26,408	38,806	+46.9%	baseline
5	32,994	98,461	+198.4%	baseline

Iteration 5 was the most extreme tuned-profile outlier. It raised the tuned profile's mean diagnostic score substantially, so the median score is the more stable condition-level summary. The conclusion does not depend only

on this outlier: `baseline-online` still had a lower diagnostic score in all five paired iterations.

Profile-level summary:

Profile	Valid runs	Median diagnostic score	Mean diagnostic score	Median frame P99	Mean scheduler >5ms
`baseline-online`	5	21,533	23,431.4	38.337ms	0.2
`kcd1-game-on-1-5-7-11-gamescope-on-0-6`	5	38,806	49,746.0	37.798ms	0.6

The `over-5ms` value is a scheduler-latency threshold count from the diagnostic score, not a frame-time threshold.

The profile-vs-profile tables above are derived from `tuning_summary.json` candidate statistics. The regenerated `tuning_recommendation.json` selects `baseline-online` as best and compares it against the best valid non-baseline candidate, `kcd1-game-on-1-5-7-11-gamescope-on-0-6`. The tuned-profile conclusion is unchanged: the tuned profile still does not validate on the current evidence.

The tuned profile was **not validated and should not be recommended on the current evidence**. The primary diagnostic score was worse in every paired iteration. Because the pair order was fixed, this should be read as a caveated non-validation result rather than a fully counterbalanced A/B estimate. The generated recommendation selected `baseline-online` as the lower-scoring profile with `NeedsRetest`, and the corrected interpretation treats ranking confidence as low once the order caveat is considered.

The careful interpretation is that the evidence did not support recommending the tuned profile, and the observed paired scores were consistently worse. This is not a claim that the profile is unsuitable across all conditions.

A/B evidence: curated copies in `evidence-bundle/kcd1/tuning_summary.json`, `evidence-bundle/kcd1/tuning_recommendation.json`, and `evidence-bundle/kcd1/tuning_recommendation.md`; original raw paths are preserved under `${KCD1_RAW}/tune/`.

10. Why the Affinity Profile Likely Failed

The profile likely reduced useful CPU capacity. KCD1/DXVK/Wine had many active worker, render, streaming, physics, audio, and helper threads. Moving the game workload from all 12 logical CPUs to `1-5,7-11` gave it only 10 logical CPUs, while reserving `0,6` for Gamescope. On this 6-core/12-thread CPU, that trade-off appears to have increased scheduling pressure more than it helped presentation isolation.

This is an interpretation, not a demonstrated causal mechanism. The evidence supporting the interpretation is:

- `profile-plan` shows important KCD worker/render/DXVK threads were moved.
- The same categories of threads appear in spike evidence.
- The tuned profile had worse diagnostic scores in every paired iteration.
- No clear GPU-utilisation explanation was needed for the profile-vs-profile result.

The lesson is not that CPU affinity is inherently a bad idea for Proton games. The lesson is that a plausible CPU-placement story still needs repeated validation, especially on a CPU where reserving one SMT pair removes a meaningful part of the available logical CPU set.

11. Statistical Interpretation

The A/B run is useful for avoiding a false positive, but it is not enough to estimate small effects precisely, and the fixed pair order further limits causal interpretation. The tool estimated that some metrics may require roughly 19-30 runs per condition because KCD1 is noisy.

Estimated runs per side for detecting a 10% movement at the observed noise level:

Metric	Estimated runs per side
`diagnostic_raw_score_total`	30
`frame_p99_ms`	30
`frame_over_16ms`	30
`frame_over_33ms`	30
`frame_over_50ms`	30
`max_latency_ns`	19

Five runs per side is enough for a case-study demonstration and enough to avoid recommending this particular false positive. It is not enough for broad tuning claims or precise small-effect estimates. This is why the generated recommendation says `NeedsRetest` even though `baseline-online` is the lower-scoring profile in this tuning run.

Uncertainty evidence: curated copy in `evidence-bundle/kcd1/tuning_recommendation.json`; original raw file is preserved under `${KCD1_RAW}/tune/`.

12. Measurement-Quality Investigation

A separate drop-counter pilot investigated the recurring non-zero wakeup replacement counter. The important finding is that the issue was not ring-buffer reserve failure.

Condition	Wakeup replacements/s	Ringbuf reserve failures
baselines	about 1436-1557/s	0
mapfactor-4 pilot	about 1568/s	0

The wakeup replacement counter was not explained by ring-buffer reserve failure, and increasing `--ebpf-wakeup-map-factor` did not reduce it. The likely interpretation is wakeup timestamp churn from rapid repeated wakeups for the same target tasks. This is documented as a measurement-quality nuance, not as dropped frame data.

Measurement-quality evidence: drop-counter pilot artifacts preserved under `${KCD1_RAW}/drop-counter-pilot/` in the raw release archive.

13. Exploratory Personal-Stack Add-On

A small exploratory comparison tested the stripped-down clean stack against the author's normal gaming configuration bundle, including `scx_lavd` and many launch flags. This changed many variables at once, so it is not a causal test of any one flag or scheduler.

Condition	Runs	Median frametime	P95	P99	Max	Median outlier %	Scheduler
clean	3	19.3419ms	26.2893ms	29.8236ms	65.7529ms	0.428%	default
personal-stack	3	20.1032ms	28.3838ms	32.4916ms	91.134ms	0.826%	`scx_lavd`

The `Max` column is the median of each condition's per-run maximum frametime, not the single worst frame observed across the condition. The worst individual personal-stack run was `personal-stack-02`, with a 181.596ms maximum frametime and a 4.814% outlier rate.

The personal stack did not show a clear advantage in this small sample. The value of this add-on is realism: `stutter` can capture and compare a complex player-used configuration bundle without overclaiming causality.

As documented in `${KCD1_RAW}/realworld-stack/ARTIFACT_NOTES.md`, the raw MangoHud CSV sources for `clean-01` and `clean-02` were not preserved when the archive was finalized. Their frame timing had already been ingested into the committed analysis JSON files, so the runs remain usable for this exploratory comparison, but the missing raw CSVs are a transparency limitation.

Exploratory evidence: real-world stack summary, notes, and launch-option artifacts preserved under `${KCD1_RAW}/realworld-stack/` in the raw release archive.

14. Limitations and Future Work

- One machine, one route, one game, one Proton version.
- Open-world workload variance is high.
- Explicit `--tree-pid` targeting was used; do not present this as a foreground auto-targeting demonstration.
- IRQ/KMS/DRM correlation was not available in the formal artifacts.
- Future KCD1 runs could use `stutter inspect-irqs`, then `--irq-latency --irq <IRQ>`.
- Future profile experiments should run `profile-plan` before A/B collection.
- A larger study would need more runs per condition.

These limitations strengthen the report if they are stated plainly. The case study is not trying to make a general performance claim about KCD1. It is showing that the method can handle a noisy real workload and avoid promoting a weak tuning story.

15. Conclusion

This case study validates the workflow rather than the tuning tweak. `stutter` successfully captured a complex KCD1/Proton workload, generated a plausible CPU-affinity hypothesis, tested it with repeated A/B measurements, and declined to recommend the hypothesis when the evidence did not support it. That ability to avoid false-positive tuning recommendations is central to reliable Linux game-performance tuning.

Appendix A: Command Shapes and Validation Commands

The case-study archive records command shapes rather than every exact shell invocation. PIDs were live process-tree roots and were re-detected before recording.

Baseline record command shape:

```
stutter record \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --duration 180 \
  --run-name kcd1-rattay-baseline-XX \
  --scenario kcd1-rattay-route-1 \
  --workload-label kcd1-proton-ge-10-34 \
  --route-label rattay-fixed-route-1 \
  --out-dir ${KCD1_RAW}/runs/baseline-XX \
  --mangohud-log <KingdomCome_MANGOHUD_CSV> \
  --hwmon \
  --cpu-freq \
  --runtime-slices \
```

```
  --foreground-window \
  --foreground-source sway
```

Profile-plan command:

```
stutter profile-plan \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profile ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --profile-name kcd1-game-on-1-5-7-11-gamescope-on-0-6
```

Explainable dry-run command:

```
stutter apply-profile \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profile ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --profile-name kcd1-game-on-1-5-7-11-gamescope-on-0-6 \
  --dry-run \
  --explain
```

Tune command shape:

```
stutter tune \
  --tree-pid <KCD1_OR_GAMESCOPE_TREE_PID> \
  --profiles ${KCD1_RAW}/profiles/kcd1-affinity-ab.toml \
  --runs 5 \
  --baseline-profile baseline-online \
  --warmup-seconds 90 \
  --epoch-seconds 270 \
  --scenario kcd1-rattay-route-1 \
  --workload-label kcd1-proton-ge-10-34 \
  --route-label rattay-fixed-route-1 \
  --mangohud-log <KingdomCome_MANGOHUD_CSV> \
  --hwmon \
  --out-dir ${KCD1_RAW}/tune/kcd1-affinity-02
```

In this tune run, each epoch used 90 seconds of warm-up followed by 180 seconds of measurement; the generated summary recorded `restore_policy = "restore-after-each"`.

Secondary fix-validation recommend command shape:

The primary profile-vs-profile conclusion is based on `${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`; the fix-validation artifacts under `${KCD1_RAW}/results/` are secondary and are marked `InvalidExperiment`.

```
stutter recommend \
  --fix-plan ${KCD1_RAW}/fix-plan-cpu-affinity-profile.json \
  --baseline ${KCD1_RAW}/runs/baseline-01 \
  --baseline ${KCD1_RAW}/runs/baseline-02 \
  --baseline ${KCD1_RAW}/runs/baseline-03 \
  --baseline ${KCD1_RAW}/runs/baseline-04 \
  --baseline ${KCD1_RAW}/runs/baseline-05 \
  --tune ${KCD1_RAW}/tune/kcd1-affinity-02 \
  --markdown ${KCD1_RAW}/results/kcd1-fix-validation.md \
  --html ${KCD1_RAW}/results/kcd1-fix-validation.html
```

JSON recommendation artifact:

```
stutter recommend \
  --fix-plan ${KCD1_RAW}/fix-plan-cpu-affinity-profile.json \
  --baseline ${KCD1_RAW}/runs/baseline-01 \
  --baseline ${KCD1_RAW}/runs/baseline-02 \
  --baseline ${KCD1_RAW}/runs/baseline-03 \
  --baseline ${KCD1_RAW}/runs/baseline-04 \
  --baseline ${KCD1_RAW}/runs/baseline-05 \
  --tune ${KCD1_RAW}/tune/kcd1-affinity-02 \
  --json \
  > ${KCD1_RAW}/results/kcd1-fix-validation.json
```

Validation checks:

```
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo fmt --all -- --check
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo test --all
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo clippy --all-targets -- -D warnings
RUSTUP_TOOLCHAIN=nightly-2026-06-06 cargo run -p xtask -- fixture-check
```

Build/check evidence: curated copies in `evidence-bundle/kcd1/build-check.txt` and `evidence-bundle/kcd1/command-output.txt`, with original setup, tune-log, and results artifacts preserved in the raw release archive.

Appendix B: Profile TOML

```
[[profile]]
name = "baseline-online"

[[profile.rules]]
affinity = "online"
match_class = ["Game", "GameHelper", "WineServer", "GameScope", "Compositor",
"Launcher", "SteamRuntime", "Helper", "Unknown"]

[[profile]]
name = "kcd1-game-on-1-5-7-11-gamescope-on-0-6"

[[profile.rules]]
affinity = "1-5,7-11"
match_comm = ["Main"]

[[profile.rules]]
affinity = "1-5,7-11"
match_class = ["Game", "GameHelper", "WineServer"]

[[profile.rules]]
affinity = "0,6"
match_class = ["GameScope", "Compositor", "Launcher", "SteamRuntime"]
```

Profile file: `${KCD1_RAW}/profiles/kcd1-affinity-ab.toml`.

Appendix C: Artifact Map

Artifact	Role
`reports/kcd1-case-study/CASE_STUDY_SUMMARY.md`	Primary archive summary
`reports/kcd1-case-study/ARTIFACT_INDEX.md`	Archive map
`${KCD1_RAW}/setup/kcd1-method-notes.md`	Route and setup notes
`${KCD1_RAW}/setup/system-info.txt`	Machine, kernel, CPU, GPU, session context
`${KCD1_RAW}/runs/baseline-*-analysis.json`	Formal baseline analysis
`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_summary.json`	Primary A/B profile comparison
`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_recommendation.html`	Generated recommendation
`${KCD1_RAW}/tune/kcd1-affinity-02/tuning_recommendation.json`	Recommendation and uncertainty data
`${KCD1_RAW}/profiles/kcd1-affinity-profile-plan-summary.json`	Profile explainability summary
`${KCD1_RAW}/profiles/kcd1-affinity-ab.toml`	Final A/B profile
`${KCD1_RAW}/drop-counter-pilot/mapfactor-4-comparison.txt`	Measurement-quality pilot summary
`${KCD1_RAW}/realworld-stack/realworld-stack-summary.csv`	Exploratory clean vs personal-stack comparison
`${KCD1_RAW}/results/kcd1-fix-validation.json`	Secondary validation artifact; status is `InvalidExperiment`

The primary source for the tuned-profile conclusion is `tuning_summary.json`. The fix-validation artifact is secondary because it compares the selected lower-scoring tune candidate, `baseline-online`, against the earlier formal baseline set and is marked `InvalidExperiment`.

Appendix D: Reproducibility Checklist

- Use the same Rattay route and save file described in the method notes.
- Use Steam with GE-Proton10-34.
- Launch KCD1 with the stripped-down measurement configuration: Gamescope and MangoHud logging, no RADV experimental flags, no FSR/FSR4, no gamemode, no mimalloc, and no forced Wine CPU topology.
- Keep `+exec user.cfg` enabled, because the archived `user.cfg` settings are part of the workload configuration.
- Use 1920x1080 through Gamescope, 100 Hz output, and a 100 FPS MangoHud cap.
- Use the same route duration: 180 seconds per measured run.
- Re-detect the live Gamescope/KCD process-tree root before each recording; do not reuse a PID from a previous launch.
- Keep background load stable: no browser, Discord stream, downloads, package builds, or other heavy activity during measurement.

- Treat hardware differences as a limitation. The recorded result is specific to this Intel i5-10600K / AMD RX 9070 XT / Wayland-Sway / Gamescope / Proton configuration.